MODUL PRAKTIK · AI DAN MACHINE LEARNING

Membangun Sistem Deteksi Jari

Kalian akan menulis seluruh JavaScript dari awal, mulai dari menghubungkan elemen HTML sampai webcam bisa menghitung jari yang terbuka.

Nama:	Kelas / No. Absen:
Tanggal:	Kelompok:

01 Tujuan

1. Menghubungkan JavaScript dengan elemen HTML melalui `id`.
2. Menjelaskan bahwa komputer membaca tangan sebagai 21 titik koordinat.
3. Menulis `for`, `if`, array, fungsi, dan operator logika untuk mengambil keputusan dari data.
4. Menguji dan memperbaiki program sendiri dengan `console.log`.

02 Persiapan dan Alur Kerja

1. Buka `index.html` di VS Code. HTML dan CSS sudah jadi. Bagian `<script>` di bawah masih **kosong**, dan di situlah kalian bekerja.
2. Jalankan dengan **Live Server** (klik kanan file, *Open with Live Server*). Jangan klik dua kali file-nya, kamera sering diblokir.
3. Tekan **F12** dan buka tab **Console** untuk melihat pesan dan error.
4. Kerjakan **Langkah 1 sampai 12** secara berurutan. Ketik sendiri, jangan copy-paste.
5. Setiap langkah punya **Letak** (di mana kode ditulis) dan **Cek** (apa yang harus terlihat). Jangan lanjut sebelum Cek terpenuhi.

> **Aturan indentasi:** setiap kali membuka `{`, geser isinya 2 spasi ke kanan. Indentasi tidak memengaruhi hasil, tetapi membuat kodemu mudah dibaca dan mudah dicari kesalahannya.

03 Konsep: Komputer Melihat 21 Titik

Model MediaPipe tidak "melihat jari" seperti manusia. Ia menemukan **21 titik** pada tangan. Tiap titik punya koordinat `x` dan `y` bernilai 0 sampai 1, disimpan dalam array `landmarks`.

```
landmarks[8].x   // posisi kiri-kanan ujung telunjuk
landmarks[8].y   // posisi atas-bawah ujung telunjuk
```

Jari	Ujung (tip)	Sendi pembanding	Sumbu
Ibu jari	4	3 (IP), 2 (MCP)	x, lalu y sebagai cadangan
Telunjuk	8	6 (PIP)	y
Tengah	12	10 (PIP)	
Manis	16	14 (PIP)	
Kelingking	20	18 (PIP)	

Titik **17** adalah pangkal kelingking, dipakai untuk memeriksa ibu jari.

> **Perhatian:** di layar komputer sumbu y dihitung dari **atas**. Nilai y yang **lebih kecil** berarti posisi **lebih tinggi**. Ini kebalikan dari grafik di pelajaran matematika.

---

04 Pemanasan Kode

Contoh ini memakai konteks lain. Pelajari polanya, lalu terapkan pada tangan.

```
const suhu  = [30, 25, 35, 27];
const batas = [28, 28, 28, 28];
let hariPanas = 0;

for (let i = 0; i < suhu.length; i++) {
  if (suhu[i] > batas[i]) {
    hariPanas++;            // sama dengan hariPanas = hariPanas + 1
  }
}
console.log(hariPanas);     // hasilnya 2

// jarak dua titik (teorema Pythagoras)
const jarak = Math.hypot(3, 4);   // hasilnya 5
```

- `const` untuk nilai tetap, `let` untuk nilai yang akan berubah.
- `for` mengulang pekerjaan untuk setiap data, `suhu[i]` mengambil data ke-`i` (hitungan mulai dari 0).
- `if` menjalankan isinya hanya jika syarat benar.
- `&&` berarti **DAN**, `||` berarti **ATAU**.
- `function nama(data) { ... }` membuat fungsi yang bisa dipanggil berkali-kali.

---

05 Peta Elemen HTML

Ini elemen di `index.html` yang akan kalian hubungkan ke JavaScript. Simpan sebagai contekan.

id	Fungsi
`webcam`	sumber gambar kamera (disembunyikan)
`output-canvas`	kanvas untuk menggambar video dan kerangka tangan

id	Fungsi
`loading-indicator`	tulisan "Memuat model..."
`fps-counter`	angka FPS
`hands-count`	jumlah tangan terdeteksi
`finger-total`	angka besar total jari terbuka
`hand-label-info`	teks "Tangan: Kanan"
`btn-toggle-cam`	tombol nyala/mati kamera
`finger-thumb`, `finger-index`, `finger-middle`, `finger-ring`, `finger-pinky`	lima baris status jari; kelas CSS `active` membuatnya menyala

---

06 Langkah Pembuatan

Langkah 1: Menghubungkan elemen HTML

`document.getElementById` mengambil elemen berdasarkan id, lalu menyimpannya di variabel agar mudah dipakai. Lima jari dikelompokkan dalam satu objek.

Letak: paling atas di dalam `<script>`.

```
const videoElement = document.getElementById('webcam');
const canvasElement = document.getElementById('output-canvas');
const canvasCtx = canvasElement.getContext('2d');
const loadingIndicator = document.getElementById('loading-indicator');
const fpsCounter = document.getElementById('fps-counter');
const handsCountEl = document.getElementById('hands-count');
const fingerTotalEl = document.getElementById('finger-total');
const handLabelInfo = document.getElementById('hand-label-info');
const btnToggleCam = document.getElementById('btn-toggle-cam');

const fingerElements = {
  thumb: document.getElementById('finger-thumb'),
  index: document.getElementById('finger-index'),
  middle: document.getElementById('finger-middle'),
  ring: document.getElementById('finger-ring'),
  pinky: document.getElementById('finger-pinky'),
};
```

Cek: ☐ Tidak ada error merah di Console saat halaman dimuat.

Langkah 2: Variabel status dan daftar jari

Variabel di atas mencatat kamera, status nyala, dan penghitung FPS. Tiga array di bawahnya sejajar: indeks ke-0 telunjuk, ke-1 tengah, dan seterusnya.

Letak: di bawah Langkah 1.

```
let camera = null;
let isCameraRunning = true;
let lastFrameTime = performance.now();
let frameCount = 0;
let fps = 0;

// Landmark ID untuk ujung jari dan sendi referensi
const FINGER_TIPS = [8, 12, 16, 20];
const FINGER_PIPS = [6, 10, 14, 18];
const FINGER_KEYS = ['index', 'middle', 'ring', 'pinky'];
```

Cek: ☐ Masih tanpa error.

Langkah 3: Fungsi `onResults`: kerangka dasar

MediaPipe memanggil fungsi ini setiap selesai memproses satu frame dan mengirim `results` berisi gambar dan data tangan. Untuk sementara fungsi ini hanya menyembunyikan tulisan "memuat", menghitung FPS, dan menggambar frame ke kanvas.

Letak: di bawah Langkah 2. Dua baris terakhir (`restore` dan kurung penutup) akan tetap berada di paling bawah fungsi, sedangkan langkah berikutnya diketik **di atas** `canvasCtx.restore();`.

```
function onResults(results) {
  // Sembunyikan loading jika pertama kali berhasil
  if (loadingIndicator.style.display !== 'none') {
    loadingIndicator.style.display = 'none';
  }

  // Hitung FPS
  frameCount++;
  const now = performance.now();
  if (now - lastFrameTime >= 1000) {
    fps = Math.round((frameCount * 1000) / (now - lastFrameTime));
    fpsCounter.textContent = fps;
    frameCount = 0;
    lastFrameTime = now;
  }

  // Sesuaikan ukuran canvas dengan frame video
  canvasElement.width = videoElement.videoWidth || 640;
  canvasElement.height = videoElement.videoHeight || 480;

  canvasCtx.save();
  canvasCtx.clearRect(0, 0, canvasElement.width, canvasElement.height);
  canvasCtx.drawImage(results.image, 0, 0, canvasElement.width, canvasElement.height);

  canvasCtx.restore();
}
```

Cek: ☐ Belum terlihat apa pun, itu normal. Fungsi ini belum dipanggil siapa pun.

Langkah 4: Menjalankan MediaPipe dan kamera

`Hands` adalah model pendeteksi tangan, `setOptions` mengatur jumlah tangan dan tingkat keyakinan, dan `hands.onResults(onResults)` mendaftarkan fungsi kita agar dipanggil tiap frame (*callback*). `Camera` mengambil gambar webcam dan mengirimnya ke model.

Letak: di bawah fungsi `onResults`.

```
// Inisialisasi MediaPipe Hands
const hands = new Hands({
  locateFile: (file) => `https://cdn.jsdelivr.net/npm/@mediapipe/hands/${file}`
});

hands.setOptions({
  maxNumHands: 2,
  modelComplexity: 1,
  minDetectionConfidence: 0.65,
  minTrackingConfidence: 0.65
});

hands.onResults(onResults);

// Inisialisasi Kamera
camera = new Camera(videoElement, {
  onFrame: async () => {
    if (isCameraRunning) {
      await hands.send({ image: videoElement });
    }
  },
  width: 1280,
  height: 720
});

camera.start().catch((err) => {
  console.error('Kamera error:', err);
  loadingIndicator.innerHTML = '<span style="color:#d84e2a;">Gagal mengakses kamera. Izinkan
akses webcam di browser Anda.</span>';
});
```

Cek: ☐ Browser meminta izin kamera

☐ Gambar kamera tampil (seperti cermin) dan tulisan "Memuat model..." hilang

☐ Angka FPS bergerak

Langkah 5: Menyiapkan penampung hasil

Setiap frame dimulai dari nol: total jari 0, belum ada nama tangan, dan semua jari dianggap tertutup.

Letak: di dalam `onResults`, tepat di bawah `drawImage` dan di atas `canvasCtx.restore();`.

```
let totalFingers = 0;
let handNames = [];
const activeFingers = {
  thumb: false,
  index: false,
  middle: false,
  ring: false,
  pinky: false
};
```

Cek: ☐ Tidak ada error, tampilan belum berubah.

Langkah 6: Kerangka perulangan tangan

Jika ada tangan terdeteksi, tampilkan jumlahnya lalu proses tiap tangan dengan `for`. Jika tidak ada, tampilkan pesan bahwa belum ada tangan. `landmarks` adalah 21 titik milik tangan ke-`i`.

Letak: di bawah Langkah 5, di atas `canvasCtx.restore();`. Komentar di dalam `for` adalah tempat kode Langkah 7 sampai 10.

```
if (results.multiHandLandmarks && results.multiHandLandmarks.length > 0) {
  handsCountEl.textContent = results.multiHandLandmarks.length;

  for (let i = 0; i < results.multiHandLandmarks.length; i++) {
    const landmarks = results.multiHandLandmarks[i];
    const handedness = results.multiHandedness[i];

    // kode Langkah 7 sampai 10 diketik di sini
  }

  handLabelInfo.textContent = `Tangan: ${handNames.join(', ')}`;
} else {
  handsCountEl.textContent = '0';
  handLabelInfo.textContent = 'Belum ada tangan terdeteksi';
}
```

Cek: ☐ Angka "Tangan Terdeteksi" berubah 0, 1, 2 saat tangan masuk dan keluar kamera.

Langkah 7: Menggambar kerangka tangan

`drawConnectors` menggambar garis antar titik dan `drawLandmarks` menggambar titiknya. Warna dan ketebalan boleh kalian ubah sesuka hati.

Letak: di dalam `for`, di bawah `const handedness`.

```
// Gambar tulang tangan dan sendi dengan gaya diagram teknis
drawConnectors(canvasCtx, landmarks, HAND_CONNECTIONS, {
  color: '#d84e2a', // oranye bata tegas
  lineWidth: 2
});
drawLandmarks(canvasCtx, landmarks, {
  color: '#151413',
  fillColor: '#eae5dc',
  lineWidth: 1.5,
  radius: 3.5
});
```

Cek: ☐ Kerangka tangan tergambar mengikuti tanganmu

Langkah 8: Mengenali tangan kiri atau kanan

Kamera ditampilkan seperti cermin, tetapi model menganggap gambarnya selfie, sehingga label `Left` dari model sebenarnya tangan kanan kita. Karena itu hasilnya dibalik dengan `isRight`.

Letak: di dalam `for`, di bawah Langkah 7.

```
// Handedness: 'Left' atau 'Right' dari perspektif model
// Karena canvas di-scaleX(-1), orientasi disesuaikan
const label = handedness ? handedness.label : 'Hand';
const isRight = label === 'Left'; // reversed due to mirror view
handNames.push(isRight ? 'Kanan' : 'Kiri');
```

Cek: ☐ Tulisan "Tangan: Kanan" muncul saat tangan kananmu di depan kamera, "Kiri" untuk tangan kiri

Langkah 9: Deteksi 4 jari

Dari pengamatan titik, jari terbuka bila ujungnya **lebih tinggi** dari sendi PIP, artinya `y` ujung **lebih kecil**. Mulai dengan satu jari dulu (telunjuk):

```
if (landmarks[8].y < landmarks[6].y) {
  activeFingers.index = true;
  totalFingers++;
}
```

Menulis `if` empat kali itu boros. Karena tiga array sejajar, satu perulangan cukup. Ganti `if` di atas dengan:

Letak: di dalam `for`, di bawah Langkah 8.

```
// 2. Deteksi 4 Jari Lainnya (Tip.y < Pip.y = jari terangkat)
for (let f = 0; f < 4; f++) {
  const tipId = FINGER_TIPS[f];
  const pipId = FINGER_PIPS[f];
  const key = FINGER_KEYS[f];

  if (landmarks[tipId].y < landmarks[pipId].y) {
    activeFingers[key] = true;
    totalFingers++;
  }
}
```

Cek: ☐ Setelah Langkah 11 selesai, empat jari menyala sesuai gerakan dan angka total berubah. Sebelum itu, buktikan dengan `console.log(totalFingers)` di bawah perulangan.

Langkah 10: Deteksi ibu jari (tantangan)

Ibu jari bergerak ke samping, jadi memakai sumbu `x`, dan arahnya berlawanan untuk tangan kiri dan kanan. Ketik bertahap:

10a. Jauh dari kelingking? Jarak ujung ibu jari ke titik 17 dibandingkan dengan jarak sendinya. Bila ujung lebih jauh, ibu jari tidak terlipat ke telapak.

```
const dTipPinky = Math.hypot(landmarks[4].x - landmarks[17].x, landmarks[4].y -
landmarks[17].y);
const dIpPinky = Math.hypot(landmarks[3].x - landmarks[17].x, landmarks[3].y -
landmarks[17].y);
```

10b. Terbuka ke samping? Tangan kanan dan kiri dibedakan dengan `isRight`.

```
const hOpen = isRight ? (landmarks[4].x < landmarks[3].x) : (landmarks[4].x >
landmarks[3].x);
```

10c. Terangkat ke atas? Untuk telapak yang menghadap kamera dengan jempol menunjuk ke atas (seperti isyarat jempol).

```
const vOpen = landmarks[4].y < landmarks[3].y && landmarks[3].y < landmarks[2].y;
```

10d. Gabungkan. Ibu jari terbuka bila ia **jauh dari kelingking DAN** (**terbuka ke samping ATAU terangkat ke atas**).

```
const thumbOpen = (dTipPinky > dIpPinky) && (hOpen || vOpen);

if (thumbOpen) {
  activeFingers.thumb = true;
  totalFingers++;
}
```

Letak: di dalam `for`, di atas atau di bawah Langkah 9 (urutannya tidak memengaruhi hasil). Kode ibu jari lengkap:

```
// 1. Deteksi Ibu Jari (Thumb)
// dTipPinky > dIpPinky memastikan jempol tidak menekuk ke dalam telapak/mengepal
const dTipPinky = Math.hypot(landmarks[4].x - landmarks[17].x, landmarks[4].y -
landmarks[17].y);
const dIpPinky = Math.hypot(landmarks[3].x - landmarks[17].x, landmarks[3].y -
landmarks[17].y);
const hOpen = isRight ? (landmarks[4].x < landmarks[3].x) : (landmarks[4].x > landmarks[3].x);
const vOpen = landmarks[4].y < landmarks[3].y && landmarks[3].y < landmarks[2].y;
const thumbOpen = (dTipPinky > dIpPinky) && (hOpen || vOpen);

if (thumbOpen) {
  activeFingers.thumb = true;
  totalFingers++;
}
```

Cek: ☐ Ibu jari terhitung di tangan kanan

☐ Ibu jari terhitung di tangan kiri

☐ Ibu jari tidak terhitung saat mengepal

Langkah 11: Menampilkan hasil ke layar

Setelah semua tangan diproses, tampilkan total jari, lalu nyalakan atau matikan lampu tiap jari dengan menambah atau menghapus kelas CSS `active`.

Letak: di dalam `onResults`, di bawah blok `if ... else` (Langkah 6) dan di atas `canvasCtx.restore();`.

```
// Update UI Tampilan Angka dan Status Jari
fingerTotalEl.textContent = totalFingers;
for (const [key, el] of Object.entries(fingerElements)) {
  if (activeFingers[key]) {
    el.classList.add('active');
  } else {
    el.classList.remove('active');
  }
}
```

Cek: ☐ Angka besar berubah sesuai jumlah jari

☐ Baris jari yang terbuka menyala

Langkah 12: Tombol nyala/mati kamera

`addEventListener('click', ...)` menjalankan kode setiap tombol diklik. Tombol bergantian antara mematikan dan menyalakan kamera sambil mengganti tulisan dan warnanya.

Letak: paling bawah di dalam `<script>`.

```
// Kontrol Tombol On/Off Kamera
btnToggleCam.addEventListener('click', () => {
  if (isCameraRunning) {
    camera.stop();
    isCameraRunning = false;
    btnToggleCam.textContent = 'Nyalakan Kamera';
    btnToggleCam.className = 'btn-secondary';
    handLabelInfo.textContent = 'Kamera Dimatikan';
  } else {
    camera.start();
    isCameraRunning = true;
    btnToggleCam.textContent = 'Matikan Kamera';
    btnToggleCam.className = 'btn-primary';
    handLabelInfo.textContent = 'Mendeteksi...';
  }
});
```

Cek: ☐ Tombol mematikan lalu menyalakan kamera, tulisan tombol ikut berganti

---

07 Uji Coba

Peragakan tiap gerakan di depan kamera, lalu isi hasilnya.

Gerakan tangan	Seharusnya	Hasil di layar	Benar?
Kepalan	0		
Telunjuk saja	1		
Simbol "peace" (V)	2		
Tiga jari (telunjuk, tengah, manis)	3		
Telapak terbuka	5		
Dua tangan terbuka	10		

Kalau hasilnya salah

Gejala	Kemungkinan penyebab
Layar kosong atau "Memuat model..." terus	Lupa Langkah 4, atau internet bermasalah (library diunduh dari internet).
Console merah: `... is not defined`	Salah ketik nama variabel (huruf besar/kecil berpengaruh) atau lupa mendefinisikannya di Langkah 1-2.
Console merah: `Unexpected token`	Kurung `(` `{` tidak berpasangan atau kurang tanda koma.

Gejala	Kemungkinan penyebab
Angka selalu 0	Kode jari ditulis di luar `for`, atau lupa `totalFingers++`.
Jari menyala saat dikepal	Tanda `<` dan `>` tertukar. Ingat y kecil berarti di atas.
Angka lebih dari 5 untuk satu tangan	`totalFingers++` berada di luar `if`.
Kamera tidak muncul	Tidak memakai Live Server, atau izin kamera ditolak.
Ibu jari terbalik kiri-kanan	Perbandingan `x` untuk `isRight` tertukar.

08 Refleksi

1. Mengapa fungsi `onResults` tidak kita panggil sendiri, tetapi cukup didaftarkan?

2. Mengapa komputer perlu membandingkan *dua titik*, bukan hanya melihat satu titik ujung jari?

3. Mengapa ibu jari memerlukan cara deteksi yang berbeda dari empat jari lainnya?

4. Apa keuntungan memakai `for` dibanding menulis `if` empat kali?

5. Kapan programmu salah menghitung? Tuliskan satu kondisi dan idemu memperbaikinya.

09 Penilaian

Aspek	Skor
Langkah 1-4: kamera dan video tampil	20
Langkah 5-8: kerangka tangan dan label kiri/kanan	15
Langkah 9: empat jari dengan `for` benar	20
Langkah 10: ibu jari benar	10

Aspek	Skor
Langkah 11-12: tampilan hasil dan tombol kamera	10
Tabel uji coba lengkap	5
Kode rapi, indentasi konsisten, ada komentar	10
Refleksi	10